

News Aggregator

A Google-News-style system — topic & publisher subscriptions, cross-publisher story clustering, general top stories, and personalized feeds at scale.

PUBLISHERS

50K+NEW ARTICLES /
DAY**5M**

MONTHLY ACTIVES

10M

FEED P99 TARGET

~100 ms

01 — PROBLEM & REQUIREMENTS

What we're building

Ingest articles from tens of thousands of publishers, **cluster the ones covering the same real-world event into a single story**, and serve two feeds: a global *Top Stories* feed for everyone and a *personalized* feed driven by each user's topic and publisher follows. We rank stories, never host full content — every click redirects to the original publisher.

FUNCTIONAL

- › Ingest / crawl from RSS, crawlers, and 3rd-party APIs
- › Cluster articles of one event → one story, keep all publisher links
- › Subscribe to topics & publishers
- › Serve global Top Stories + personalized feed
- › Redirect to original article on click
- › Free-text search across articles

NON-FUNCTIONAL

- › **Low read latency** — ~100 ms P99 feed serve
- › Read-heavy; reads » writes
- › Freshness — breaking news visible in seconds–minutes
- › Ingest spikes on breaking news (diurnal + bursty)
- › Availability > strict consistency (eventual OK)
- › Horizontally scalable ingestion & serving

● SCOPE THE INTERVIEW FIRST

The prompt drifts two ways — some loops assume articles already exist and focus on **ranking / trending**; others want full **provider crawling & APIs**. Ask early. Also confirm the three explicit evaluation axes you're being graded on: **ingestion pipelines, caching, and personalization at scale** — and steer depth there.

02 — CAPACITY ESTIMATION

Back-of-envelope, from the anchors

Everything below derives from three numbers: **50K publishers, 5M articles/day, 10M MAU**. State assumptions out loud — the numbers matter less than the reasoning.

INGEST WRITE QPS

~**58**/s avg

~1K/s breaking-news peak

FAN-OUT (PUSH) WRITES

~**116K**/s

why pure push breaks

FEED READ QPS

~**230**/s avg

~2K/s peak

METADATA STORAGE

~**3.7**TB/yr

payloads to object store

HOT CACHE WORKING SET

<**1**GB

→ replicate everywhere

METADATA PARTITIONS

~**32**

by publisherId, room to grow

1 • Ingestion write rate

5,000,000 articles / 86,400 s ≈ **58 writes/s** avg

breaking-news spike ×10–15 ≈ **600–900/s**

→ design ingestion for **~1K writes/s** sustained headroom

2 • Fan-out-on-write cost (the reason for hybrid)

assume avg 10 follows/user $\rightarrow 10M \times 10 = 100M$ follow edges
 avg followers/publisher = $100M / 50K = 2,000$
 push writes = $58 \text{ art/s} \times 2,000 \approx 116K$ feed-writes/s
 $\rightarrow$ celebrity publishers (millions of followers) blow this up $\rightarrow$ **hybrid**

3 • Read QPS

DAU $\approx 20\%$ of MAU = 2M
 ~ 10 feed fetches/DAU/day $\rightarrow 20M$ reads/day $\approx 230/s$ avg
 peak $\times 8-10 \approx \sim 2K$ reads/s
 $\rightarrow$ reads $\gg$ feed-store writes on a per-user basis; **optimize the read path**

4 • Storage

metadata $\sim 2KB/article \times 5M = 10GB/day \rightarrow \sim 3.7TB/yr$
 raw text/payload $\sim 30KB \times 5M = 150GB/day \rightarrow$ object store / doc DB
 embedding 384-d f32 $\approx 1.5KB \times 5M = 7.5GB/day \rightarrow$ ANN over recent window
 serving store keeps a hot **30-90 day** window ($\sim 0.3-0.9TB$), rest archived

5 • Cache memory — the key insight

hot set = active clusters, not all articles
 $\sim 200K$ hot clusters $\times \sim 4KB$ (with member list) $\approx 0.8GB$
 precomputed feeds: $2M \text{ DAU} \times 200 \text{ cluster-IDs} \times 8B \approx 3.2GB$ (IDs only)
 $\rightarrow$ hot working set fits in RAM $\rightarrow$ **replicate the article/cluster cache on every node** instead of sharding

6 • Partitions & the 100 ms budget

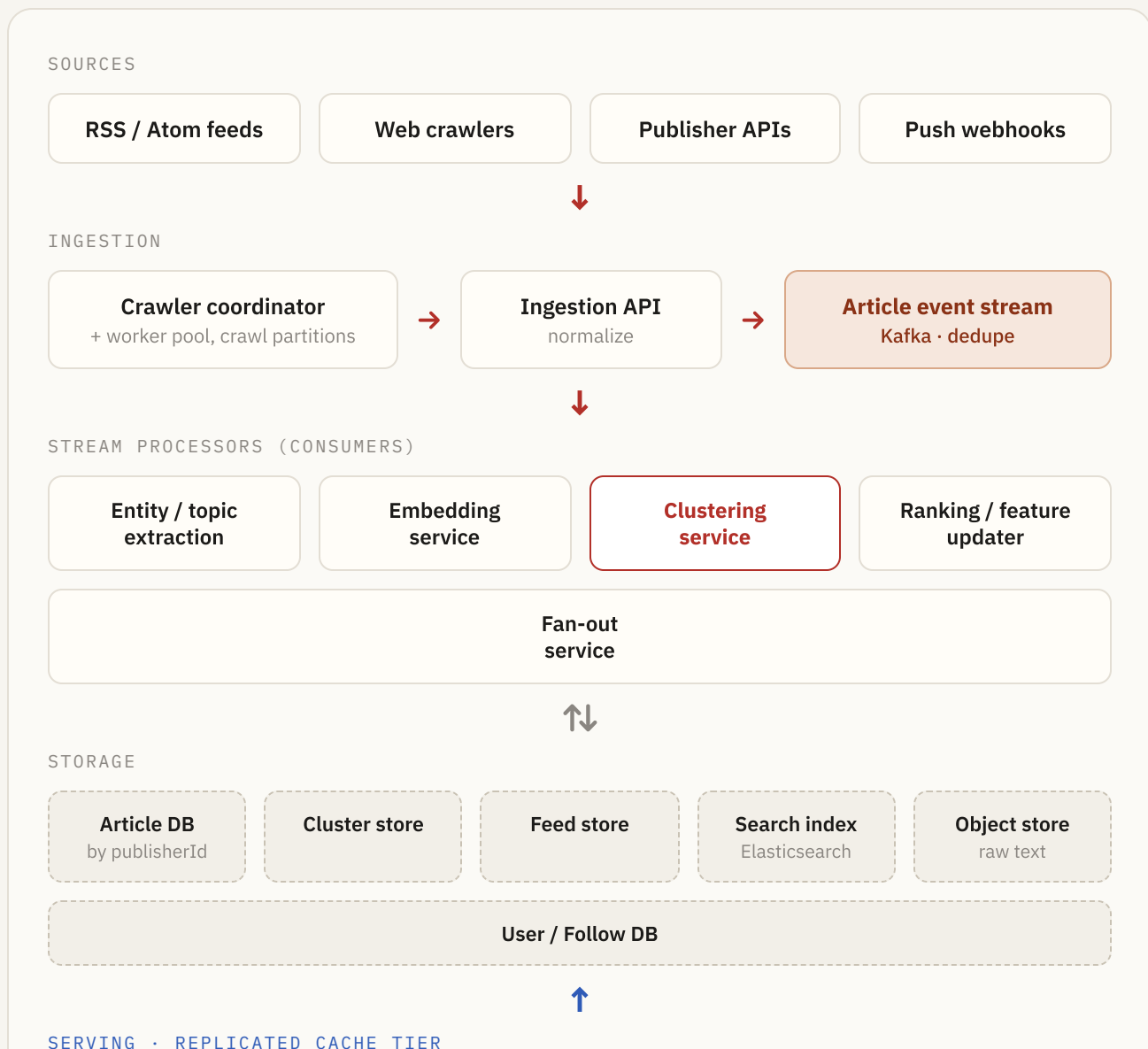
metadata by publisherId: $3.7TB/yr \div \sim 0.5TB/shard \approx 8$, use ~ 32 for parallelism
 Kafka: $1K/s$ peak, clustering $\sim 50ms/article \rightarrow \sim 50$ workers $\rightarrow \sim 64$ partitions
 read budget: API 5 + feed-ID cache 2 + hydrate 200 clusters from RAM 8 +
 celebrity merge 10 + rank/trim 6 + serialize 5 $\approx 36ms \rightarrow$ comfortably under 100ms if nothing hot-path touches disk

03 — HIGH-LEVEL ARCHITECTURE

The whole system on one page

An event-streamed pipeline. Articles flow in on the left, get normalized → enriched → clustered, then land in purpose-built stores. The read path never touches those stores directly — it's served entirely from a replicated cache tier to hit the ~100 ms budget.

— write / ingest path — read / serve path





● THE ONE-LINER

“Ingestion is a **stream**; stores are **materialized views** off that stream; serving is a **cache** in front of those views.” Each store exists because one query pattern needs it — don't force one database to do all six jobs.

API — DESIGN

The read & write contract

A thin, cache-friendly REST surface over the serving tier. Feeds are **cursor-paginated** (never offset — the list shifts constantly), reads are cheap & cacheable, and the only true “write” a client makes is a follow or a lightweight event beacon.

VERB	PATH	PURPOSE
GET	/v1/feed	Personalized feed for the authed user (cursor)
GET	/v1/top-stories	Global feed; ?topic=&country=, no auth needed
GET	/v1/stories/{clusterId}	One story + all publisher versions behind it
GET	/v1/search	?q= free-text; cluster-collapsed results
GET	/v1/topics	Topic catalog for the subscribe UI
POST	/v1/follows	Follow a publisher/topic · DELETE /v1/follows/{id}

POST	/v1/events	Batched read/click/dwell beacons (fire-and-forget)
GET	/v1/redirect	?articleId= logs click, 302 to publisher

```
GET /v1/feed?cursor=...&limit=20

{
  "stories": [
    {
      "clusterId": "c_9f3",
      "title": "...",
      "topics": ["politics"],
      "sourceCount": 18,
      "repPublisher": "...",
      "updatedAt": "...",
      "sources": [{ "publisher": "...",
                    "articleId": "a_1", "url": "..."}]
    }
  ],
  "nextCursor": "eyJzY29yZSI6..."
}
```

```
POST /v1/events

{
  "events": [
    { "type": "read", "clusterId":
      "c_9f3",
        "dwellMs": 4200, "ts": "..."} ,
    { "type": "click", "articleId":
      "a_1",
        "publisherId": "p_7", "ts": "..."}
  ]
}
→ 202 Accepted    // dropped on the
                  // event stream, non-
                  blocking
```

Design principles

Cursor pagination encodes (score, id) so an inserted breaking story never duplicates/skips rows. **Versioned** path prefix. **Idempotent** follows (PUT-like). Reads carry Cache-Control + ETag for edge/CDN caching.

Why a redirect endpoint

We never host content, so click-through goes through /v1/redirect: it records the click (feeding aggregation & ranking) then 302s to the publisher — one place to attribute traffic and enforce per-publisher click accounting.

Getting articles in

Three inbound modes, all normalized into one **article event** and dropped on Kafka. Kafka is the shock absorber — it decouples bursty ingestion from steadier downstream processing and gives replay for reprocessing (e.g. when the clustering model changes).

PULL — RSS / CRAWL

Poll feeds & crawl on a schedule. Simple, works for everyone, but latency = poll interval and wastes fetches on quiet feeds.

PUSH — WEBHOOKS / PUBSUBHUBBUB

Publisher notifies us on publish. Low latency, no wasted polls — but only big publishers support it. Adaptive poll rate for the rest.

3RD-PARTY APIS

Licensed news APIs / wire services. Clean structured data, rate-limited — wrap each in an adapter that emits the same canonical event.

Crawler coordination



Single coordinator

Simple to start — hands crawl partitions to workers, tracks liveness. **But it's a single failure domain** and a scaling bottleneck.

Multi-coordinator

Leader election / membership (ZooKeeper, etcd, or consistent-hash ring). **Clear ownership of crawl partitions** so no two workers fetch the same source.

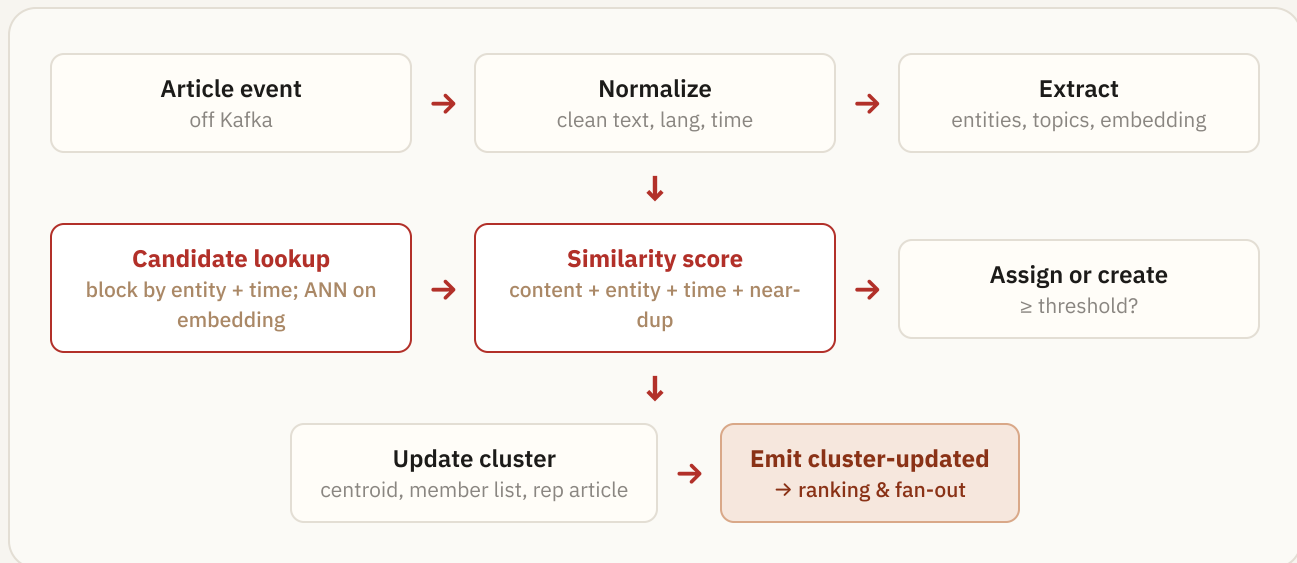
● DEDUPE AT THE DOOR

Canonicalize URLs (strip tracking params, follow redirects, resolve AMP/syndication) and hash content. A syndicated wire story appears under dozens of URLs — kill exact/near duplicates *before* Kafka so clustering isn't flooded with copies. Keep a short-TTL seen-URL set (Redis/Bloom filter) keyed on canonical URL + content hash.

05 — STORY CLUSTERING

Many articles, one story

A first-class design axis. Twenty outlets cover the same election result — we want **one story entry** that links to all twenty. Keep both records: the `Article` (points to the publisher URL for click-through) and the `StoryCluster` (what ranking & feeds operate on).



Why blocking first

Never score against every open cluster. **Block** the candidate set by shared entities and a time window (events are local in time), then run ANN over embeddings only within that block. Turns an $O(N)$ scan into a bounded lookup.

The similarity signal

Weighted blend: embedding cosine + named-entity overlap (Jaccard) + timestamp proximity + title/near-dup (MinHash/SimHash). One learned threshold decides join-vs-new.

● ACCURACY: THE TWO FAILURE MODES

Over-clustering (threshold too loose) merges distinct events — two shootings become one story. **Under-clustering** (too tight) fragments one event into many. Tune the threshold on a labeled set; report precision/recall, not accuracy.

Time-decay clusters: stop admitting new members after a cluster goes cold, so a fresh event about the same entities (“another quake in the same city”) forms its own story instead of being absorbed by yesterday's.

06 — DATA MODEL & SCHEMAS

Entities, keys, indexes

Split the answer by entity when pushed for a concrete DB choice — don't hand-wave “relational or document, either works.” **PK** = partition key, **IDX** = secondary index.

Article

```

articleId      uuid    // IDX
publisherId    uuid    // PK
clusterId      uuid    // IDX
canonicalUrl   text
title, snippet text
publishedAt    ts      // IDX
entities[]     text
topics[]       text
embeddingRef   uuid
  
```

StoryCluster

```

clusterId      uuid    // PK
repArticleId   uuid
memberIds[]    uuid    // all publishers
topics[]       text    // IDX
centroid       vector
firstSeenAt    ts
lastUpdatedAt  ts      // IDX
score          float   // IDX
sourceCount    int
  
```

Publisher

```

publisherId    uuid    // PK
name, domain   text
qualityScore   float
followerCount  int     // offline job
isCelebrity    bool    // IDX
crawlPolicy    json
  
```

Subscription / Follow

```

userId         uuid    // PK
targetType     enum    //
publisher|topic
targetId       uuid
weight         float   // implicit
signal
createdAt      ts
// reverse IDX: targetId -> userId//
(fan-out needs both directions)
  
```

Topic

```

topicId        uuid    // PK
name, slug     text
parentId       uuid    // hierarchy
followerCount  int
  
```

Read / Click event



```
eventId      uuid
userId       uuid
articleId    uuid
clusterId    uuid
publisherId  uuid  // for click-agg
type         enum  // read|click
ts           ts
// append-only event stream, not a
table
```

Store by job

DATA	STORE	WHY
Users, follows, publishers, crawl-job leases	Relational	Transactions, joins, normalized; job leases need atomic claim
Article metadata	Relational / wide-column	Partition by publisherId; high write volume, simple lookups
Raw payload + extracted text	Object / document store	Big blobs, write-once, rarely read in hot path
Free-text queries	Search index (ES)	Inverted index, relevance scoring, facets
Embeddings	Vector / ANN index	Cluster candidate lookup over a recent window
Precomputed feeds, hot clusters	Redis / in-memory cache	Sub-ms reads; the whole serving path lives here

The central trade-off

How does a followed publisher's new story reach a subscriber's feed? Precompute it on publish (**push / fan-out on write**) or assemble it at request time (**pull / fan-out on read**)? Neither wins outright — the standard answer is a **hybrid**.

Push — fan-out on write

On publish, write the story into every subscriber's precomputed feed list.

- + reads are trivial (one key lookup)
- + cost paid off the hot path
- write amplification = follower count
- celebrity publish = millions of writes

Pull — fan-out on read

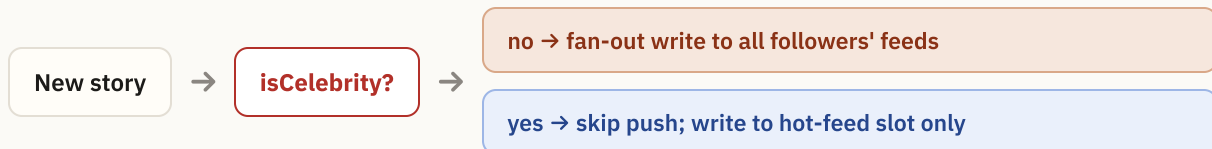
At read time, gather recent stories across everything the user follows and merge.

- + writes are cheap (one copy)
- + no wasted work for inactive users
- read latency grows with # follows
- heavy work on every request

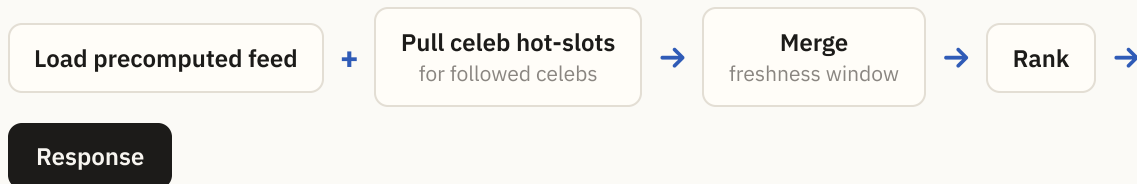
The hybrid — celebrity-publisher path

Push for normal publishers; **skip** push for celebrities and pull them in at read time, merged into the precomputed feed within a freshness window.

ON PUBLISH



ON READ



Identifying celebrities

A **followerCount threshold** on the publisher row, maintained by an offline job. Cross it → set `isCelebrity`. The threshold is a **tuning knob**: lower it to cut write amplification, raise it to cut read-merge cost.

Why the hot-feed slot works

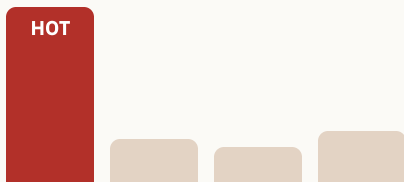
There are few celebrities and they publish often, so their latest stories are cached once and read by millions — a great cache-hit ratio. It's the *same replicated cache* pattern used for hot articles.

08 — CACHING STRATEGY

Replicate hot, don't shard it

News is **viral and skewed**: a handful of breaking stories take a huge share of reads. Sharding the cache by `articleId` puts each viral article on one node — and hammers it. Since the hot working set is under ~1 GB (see §02), just replicate it.

Sharded by `articleId` — hotspots



One shard melts on the viral article; the rest idle.
Load impossible to balance.

Replicated — any node serves any article



Reads spread evenly; add nodes for more read throughput. Cost: more RAM, slightly colder warm-up.

WHAT'S CACHED

Hot cluster records + rep articles, precomputed feed ID-lists, celeb hot-slots, Top-Stories list. Store IDs in feeds; hydrate from the cluster cache.

INVALIDATION

Short TTL (news ages fast) + event-driven bust on cluster-updated. Stale-for-seconds is acceptable here.

NOT IN THE DB

Dedicated cache tier, not “query the DB and hope the buffer pool holds it.” The hot path must never touch disk to make 100 ms.

● CLICK AGGREGATION — TWO-LEVEL TOP-K

Click/read events stream in partitioned by `publisherId` (or `clusterId`). Each partition keeps a local **top-K**; a merge step combines per-partition top-Ks into a **global top-K**. You avoid a global counter hotspot, and the merged signal feeds ranking, trending, and the offline followerCount / celebrity job.

09 — RANKING & HOT-TOPIC DETECTION

Ranking clustered stories

Rank **clusters**, not articles. Four axes named in the prompt — freshness, user relevance, source quality, and source diversity — blended into one score, recomputed on **cluster-updated** and cached.

```
SCORE(CLUSTER, USER)

= w1 · freshness( $\Delta t$ ) + w2 · relevance(user, topics)
  + w3 · quality(sources) + w4 · diversity(distinct pubs) +
  w5 · engagement(click/read velocity)
```

■ Freshness

Exponential time-decay on the cluster's latest update. Half-life in hours — news dies fast.

■ Relevance

Overlap of cluster topics/entities with the user's follows & learned preferences.

■ Source quality

Per-publisher trust/authority score. Boost reputable originators, demote content farms.

■ Source diversity

Reward clusters covered by many independent publishers — corroboration signals a real, significant event.

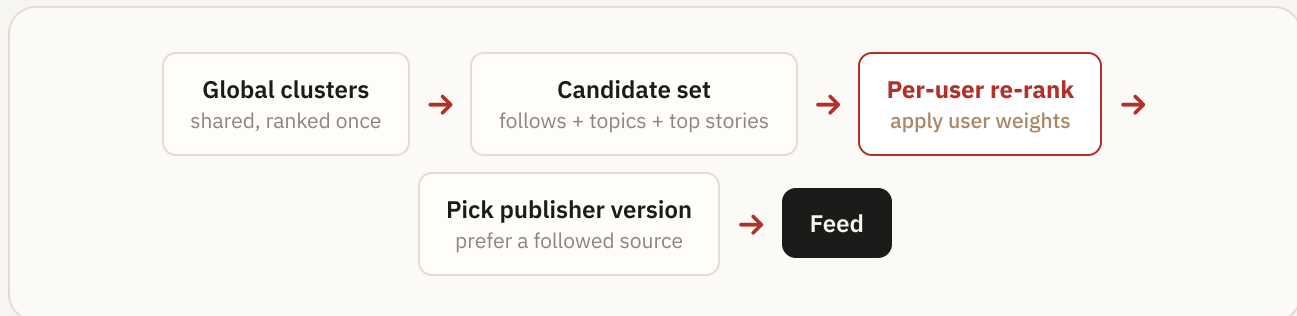
● HOT-TOPIC / TRENDING DETECTION

A streaming ranking problem over *fresh* clusters. Score by **article-count velocity** (many publishers piling on fast), **source diversity**, **click/read velocity**, and **freshness decay**. Promote a cluster to Top Stories / trending when its score crosses a threshold. Velocity — rate of change, not raw count — is what separates a breaking event from an evergreen popular story.

10 — PERSONALIZATION AT SCALE

Personalize after clustering

Order of operations matters: **cluster first** (event-level, shared by everyone), **then personalize** by re-ranking those clusters per user. This keeps expensive clustering global — done once — while personalization is a cheap per-user re-rank of an already-small candidate set.



Signals

Explicit: publisher follows, topic subscriptions.

Implicit: clicks, reads, dwell time, dismisses — folded into a per-user topic/publisher affinity vector by an offline job.

Cold start

No history → serve global Top Stories + onboarding topic picks. Blend toward personalization as signals accumulate. Personalized feed = Top Stories when the affinity vector is empty.

● WHY RE-RANK, NOT RE-CLUSTER

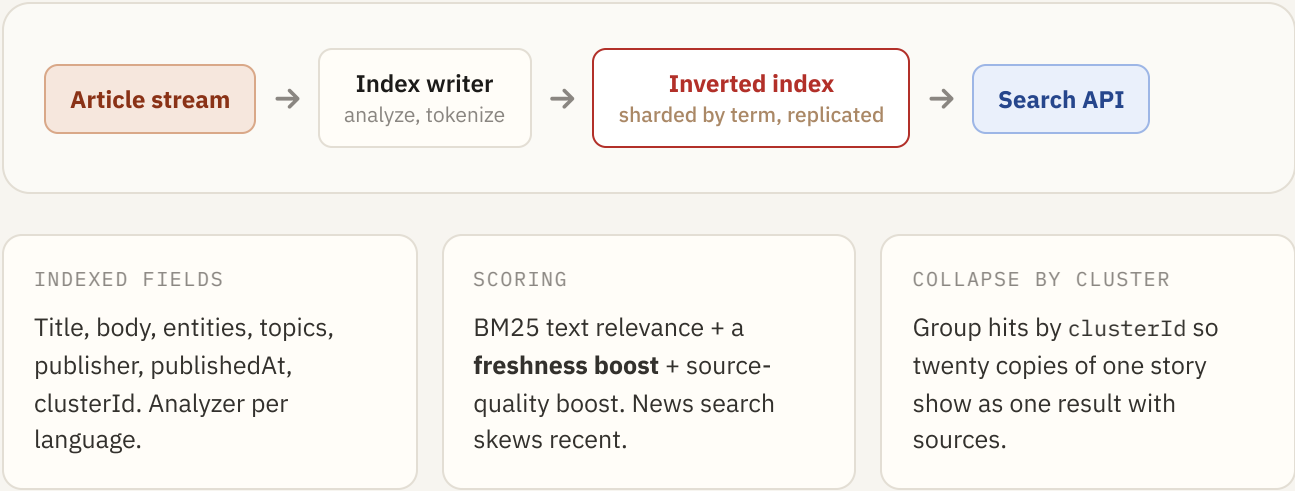
The precomputed (push) feed holds **cluster IDs**. The user's affinity vector is small and cached. Re-ranking a few hundred candidates against it is microseconds — it fits the 100 ms budget without any per-user clustering work.

Diversity guardrail: cap stories per topic/publisher and inject some Top Stories so the feed doesn't collapse into a filter bubble.

11 — SEARCH INDEX

Free-text search

A **separate inverted index** (Elasticsearch-style) built off the same article stream — not the serving DB. Search and the ranked feed are queried **in parallel** and merged for personalized search results.



FR — FAILURE & RECOVERY

When things break

The event-stream backbone makes most failures **recoverable by replay** rather than data loss. The read path degrades gracefully — a stale-but-fast feed beats a fresh-but-down one. Walk each tier and name its blast radius.

FAILS	IMPACT	RECOVERY
Crawler worker	A crawl partition stalls	Coordinator reassigns partition to a live worker; lease expires

Coordinator	No new crawl assignments	Standby wins leader election (etcd/ZK); workers keep current leases
Kafka broker	Partition leader loss	ISR replica takes over; producers retry idempotently (no dupes)
Clustering consumer	Enrichment backs up	Consumer-group rebalance; resume from committed offset — lag, not loss
Cache node	Read misses spike briefly	Replicated tier — peers still serve; rehydrate from stores
Whole cache tier	Thundering herd on stores	Request coalescing + jittered TTL; rebuild feed store from event log
Region	Users in-region down	DNS/anycast failover; async cross-region replication of stores

REPLAY AS RECOVERY

Stores are materialized views off Kafka. Corrupt a cluster store or ship a bad model? Rewind the offset and rebuild — the log is the source of truth (bounded by retention).

IDEMPOTENCY

At-least-once delivery means retries. Key writes on `articleId` / canonical URL so a replayed event is a no-op, not a duplicate story.

GRACEFUL DEGRADATION

Personalization down → serve Top Stories. Search down → feed still loads. Never let a non-critical dependency take the feed with it.

● POISON MESSAGES & DLQ

One malformed article shouldn't wedge a consumer partition and stall everything behind it. Bound retries, then route the failure to a **dead-letter queue** for offline inspection and keep the stream moving. Pair with alerting on consumer **lag** — it's the earliest signal that ingestion is falling behind a breaking-news spike.

Defend each choice against its opposite

The interviewer will push the alternative on every decision. Have the one-line defense ready.

DECISION	WHY	INSTEAD OF
Hybrid push/pull	Push is cheap to read; pull avoids celebrity write storms	Pure push (write amp) / pure pull (read latency)
Replicated cache	Viral articles would hotspot one shard	Shard by articleId
Dedicated cache tier	Hot path must never touch disk for 100ms P99	Rely on DB buffer pool
Cluster then personalize	Clustering is global & costly; re-rank is cheap per user	Per-user clustering
Kafka between ingest & process	Absorbs breaking-news spikes; enables replay	Synchronous write to DB
Separate search index	Inverted index $\neq$ serving store's access pattern	LIKE queries on the article DB
Store both Article + Cluster	Rank on clusters; click-through needs the publisher URL	Only keep clusters
Two-level top-K	No global counter hotspot; scales with partitions	Single global counter

Gotchas to name before you're asked

› Dedupe syndicated wire stories *before* clustering

› Crawler coordinator is a failure domain — make it HA

› Time-decay clusters so new events don't merge into old

› Freshness window on the celeb read-merge

› Diversity guardrail against filter bubbles

› Respect robots.txt & publisher rate limits when crawling

■ NEWS AGGREGATOR — SYSTEM DESIGN ———— Defend every choice against its opposite.